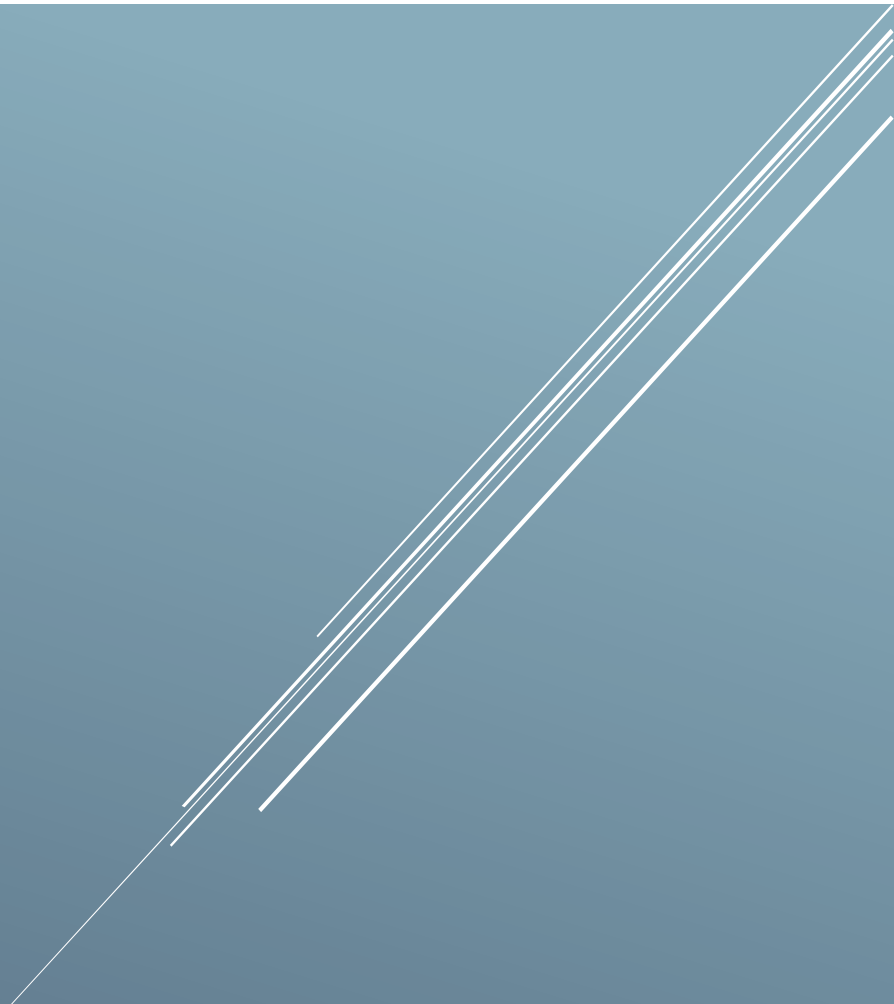




DESEÑO DE SOFTWARE

Resumo Final

- 1.Introducción
2. Modelado de casos de uso
- 3.Modelado Estructural
- 4.Modelado de comportamiento
5. Modelado Estructural Avanzado
6. Patróns de Deseño
7. Patróns de Creación
- 8.Patróns Estructurais
9. BONUS

1.Introdución

Que é un Modelo?

- É unha **simplificación** da realidade
- Proporciona **planos** dun sistema en maior ou menor detalle
- Un bo modelo é adecuado ao nivel de **abstracción** requirido
- Dous tipos de modelo
 - o **Estruturais**: destacan a organización do sistema
 - o **De comportamento**: resaltan a súa dinámica

Estratexias de creación de modelos

- **Enxeñaría directa**: xeración de código a partir dun modelo
- **Enxeñaría inversa**: reconstrución dun modelo a partir da implementación (require intervención humana)

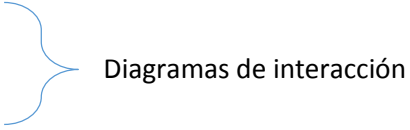
Tipos de diagramas en UML

1.x – 2.x

- Parte estática do sistema

1. Diagramas de clases
2. Diagramas de obxectos
3. Diagramas de compoñentes
4. Diagramas de despliegue
5. Diagramas de paquetes

- Parte dinámica do sistema

5. Diagrama de casos de uso
 6. Diagramas de secuencia
 7. Diagramas de colaboración
 - ~~7. Diagramas de colaboración~~
 7. Diagramas de comunicación
 8. Diagramas de estados
 9. Diagramas de actividades
 10. Temporización
 11. Estrutura composta
 12. Vista de iteración
 13. Perfil
- 
- Diagramas de interacción

2. Modelado de casos de uso

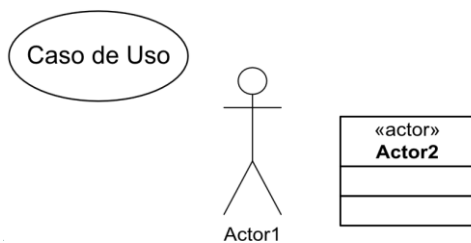
Funcionalidades

- Capturan comportamiento
- Especifican requisitos funcionais
- Permiten comunicación entre diferentes persoas dun equipo de desenvolvemento

Descricións

- Casos de uso:
 - Conxunto de secuencias de accións que executa un sistema para producir un resultado de beneficio para un actor.
 - Conxunto de escenarios ligados por un obxectivo de usuario común.
- Escenario:
 - Secuencia específica de accións que describe unha iteración entre un usuario e o sistema.
 - Os escenarios son instancias dos casos de uso.
- Secuencia: iteración do sistema con elementos externos
- Actor: rol que xoga un usuario ou sistema externo ao que se está a modelar.

Representación



Especificación de casos de uso

- Especificanse textualmente como un fluxo de eventos
 - Escenario principal
 - Fluxos alternativos
- Granularidade a elección do deseñador (*dependente da complexidade da acción*)
- Non ten correlación coas clases do sistema

Exemplo Especificación

Comprar un producto

1. El cliente selecciona artículos del catálogo
2. El cliente va a la caja
3. El cliente introduce la información de reparto
4. El sistema muestra el precio final
5. El cliente introduce los datos de su tarjeta
6. El sistema autoriza la venta
7. El sistema confirma la transacción

Alternativa nº 1: Fallo en autorización (paso 6)

Permitir reintroducir los datos de tarjeta (paso 5)

Alternativa nº 2: Cliente habitual

Se muestran valores por defecto de reparto y tarjeta

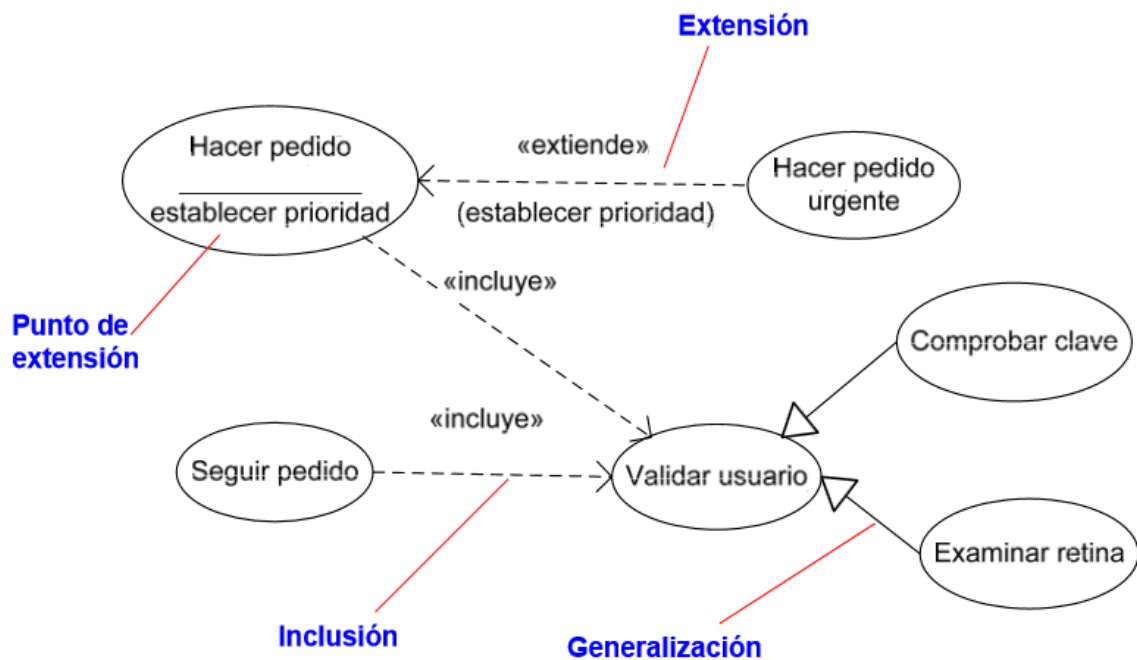
El cliente acepta o modifica datos y salta al paso 6

Diagramas de casos de uso

- Representan comportamiento
- Empréganse para
 - o Modelar o contexto dun sistema
 - Modelar o contexto é dicir que elementos pertencen ao sistema
 - Os elementos son responsables de levar a cabo comportamento
 - Contexto: está formado por elementos externos (actores) que traballan contra o sistema
 - o Modelar os requisitos funcionais
 - Modelar requisitos implica especificar que debe facer o sistema
 - Requisito: característica de deseño, propiedade ou comportamento do sistema.
 - Un requisito é un contrato entre o sistema e elementos externos
- Non se empregan para:
 - o **Mostrar estrutura**
 - o **Mostrar fluxo**
 - o **Mostrar despliegue**
- Elementos
 - o Actores: roles que empregan os usuarios ou sistemas externos
 - Débense indicar os que obteñen algún beneficio do caso de uso.
 - o Casos de uso
 - Poden non estar ligados a ningún actor
 - o Asociacións entre actores e casos de uso
 - o Relacións
 - Xeneralización: O caso de uso fillo herda o comportamento do pai e pode engadir ou redefinir comportamento. (*emprégase sobre todo para substituír unha funcionalidade por outra máis completa*)
 - Inclusión: Un caso de uso incorpora o uso de outro. (*emprégase sobre todo para extraer en novos casos de uso operacións que se repiten en moitos deles por separado*)
 - Extensión: Un caso de uso modifica o comportamento pero só en certos puntos de extensión. (*o seu emprego é similar ao da xeneralización, pero este emprégase sobre todo cando queremos*

crear unha variante da mesma funcionalidade –algo así como un strategy-)

Exemplo



3.Modelado Estrutural

Clases

- Descripción dun conxunto de obxectos que comparten atributos, operacións, relacións e semántica.
- Son abstraccións independentes da linguaxe de programación e son capaces de implementar unha ou máis interfaces.
- Elementos:
 - o Nome: único dentro do paquete para cada clase
 - o Atributos: propiedades compartidas para todos os obxectos da clase
 - Definen o estado
 - Sintaxe: visibilidade nome : tipo = valor por defecto
 - o Operacións: servizos que poden ser requiridos a calquera obxecto da clase
 - Determinan o comportamento
 - Sintaxe: visibilidade nome (parámetros) : tipo retorno
- Responsabilidades:
 - o Contrato de unha clase
 - o Definidos polos atributos e as operacións
 - o A maioría de clases colaboran con outras para satisfacer as súas responsabilidades.

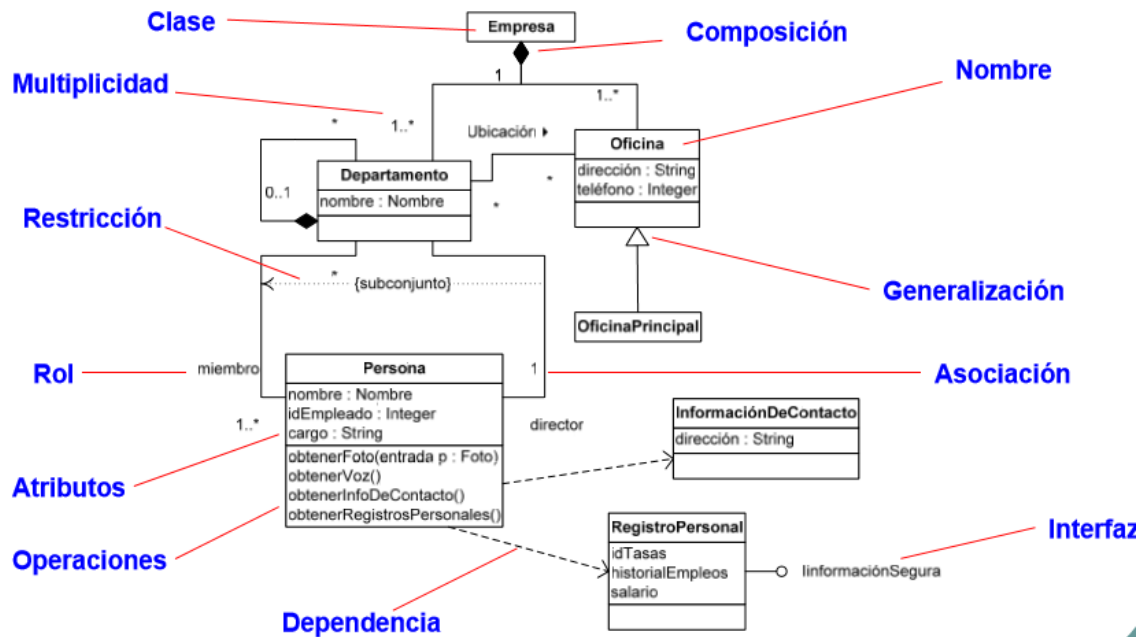
Relacións

- Establecen colaboracións entre as clases
- Tipos de relacións:
 - Dependencia
 - Relación de uso
 - Declara que os cambios na especificación dun elemento poden afectar a outro que o empregue.
 - Emprégase para indicar utilización
 - Xeneralización
 - Relación entre un elemento xeral (pai) e outro específico (fillo)
 - Os fillos poden substituír ao pai pero o pai non aos fillos
 - Polimorfismo
 - Herdanza múltiple.
 - Asociación
 - Relación estrutural entre obxectos
 - Relacionan clases do mesmo nivel
 - Adornos:
 - Nome: etiqueta que describe a relación
 - Rol: papel que xoga cada unha das clases
 - Multiplicidade: nº de instancias participantes
 - Agregación: asociación todo-parte
 - Notas: engaden ao modelo requisitos, observacións, revisións e explicacións.
 - Especificar asociación entre dúas clases se:
 - Necesítase navegar entre os obxectos (*unha clase é un atributo doutra*)
 - As súas instancias interactúan entre elas.
 - Empregar agregación se unha se percibe como un todo fronte á outra.

Diagramas de Clases

- Columna vertebral do modelado Orientado a Obxectos
 - Clases, interfaces e colaboracións
 - Relacións
 - Dependencias
 - Xeneralizacións
 - Asociacións
 - Realizacións
 - Modelan a vista do deseño
 - Base para diagramas de paquetes, compoñentes e despliegue
 - Usos:
 - Modelo de vocabulario dun sistema (*que abstraccións forman o sistema e cales son as súas responsabilidades*)
 - Estrutura de colaboracións
 - Esquema conceptual dunha Base de Datos

Exemplo de Diagrama de Clases



Tarjetas CRC (Clase –Responsabilidad-Colaboración)

- Non forman parte de UML
- Partes:
 - o Nome: Nome da clase
 - o Responsabilidade: frases cortas que resumen o que deberían facer os obxectos.
 - o Colaboracións: resto de clases nas que se apoia.
- Vantaxes:
 - o Fomentan a discusión
 - o Permiten pensar en interaccións entre obxectos sen empregar diagramas
 - o Evitan crear clases que sexan meros contedores de datos (*polo uso das responsabilidades*)
- Risco:
 - o Xerar grandes listas de responsabilidades de baixo nivel

Exemplo tarjetas CRC

Pedido	
Responsabilidades	Colaboraciones
Comprobar artículos en stock	LineaPedido
Determinar precio	Cliente
Comprobar validez de pago	
Enviar a dirección de reparto	

4. Modelado de comportamiento

Iteracións

- Comportamento que inclúe mensaxes intercambiados por un conxunto de obxectos dentro dun contexto para lograr un propósito.
- Modelan dinámica de colaboracións
- Colaboracións: Sociedades de obxectos que proporcionan un comportamento maior que a suma dos seus comportamentos
- Contexto:
 - As iteracións serven para modelar fluxo de control en:
 - Un sistema ou un subsistema
 - A implementación dunha operación
 - Unha clase ou un compoñente
 - Os obxectos dunha iteración poden ser concretos ou prototípicos
 - Poden aparecer instancias de interfaces e clases abstractas.

Mensaxes

- Transmisión de información entre obxectos coa finalidade de desencadear unha actividade.
- Tipos:
 - Chamada: Invocación de operacións
 - Retorno: Devolución dun valor
 - Envío de sinais
 - Creación de obxectos
 - Destrucción de obxectos

Enlaces

- Conexión semántica entre obxectos pola que se transmite unha mensaxe
- Xeralmente instancias dunha asociación
- Adornos para o extremo dun enlace:
 - <<association>> existencia de asociación
 - <<self>> invocador dunha operación
 - <<global>> accesible por ter alcance global
 - <<local>> visible por estar declarado localmente
 - <<parameter>> visible por ser un parámetro

Secuencias

- Fluxo de mensaxes encadeados entre diferentes obxectos
- Toda secuencia se orixina dentro dun proceso ou fío e se executa mentres este non finaliza.
- Cada proceso define un fluxo de control separado onde as mensaxes se ordenan segundo se sucedan no tempo

Diagramas de Interacción

- Describen a forma na que colaboran distintos obxectos para producir un comportamento.
- Conteñen obxectos, enlaces e mensaxes.
- Síncronos e asíncronos
 - o Síncronos -> Cabezas de frechas sen estar recheas
 - o Asíncronos -> Cabezas de frechas están recheas (*filled*)
 - o *Hai que ter coidado ao lelas, xa que moitas veces o autor do diagrama non ten isto en conta á hora de deseñalo. Polo tanto non podemos asumir que isto se cumpre a non ser que saibamos seguro que o está a empregar.
- Mostran secuencia dinámica de mensaxes que flúen polos enlaces.
- Tenden a recoller comportamento dun só caso de uso
- Dous tipos equivalentes
 - o Diagramas de secuencia
 - Destaca a organización temporal das mensaxes
 - Liña de vida: Representa a existencia dun obxecto ao longo dun período de tempo
 - Foco de control: Representa o tempo durante o cal o obxecto executa unha acción (*os cadradiños das liñas de vida*)
 - En UML 1 empregábanse obxectos (polo tanto nomes subliñados) en UML 2 xa non se empregan polo tanto deben ir sen subliñar.
 - o Diagramas de colaboración (De comunicación)
 - Destaca a organización estrutural dos obxectos
 - Enlace: pódese asociar un estereotipo de camiño aos extremos (omitírase <<association>>)
 - Nº de secuencia: numeración decimal para mostrar anidamento
- Iteración e Bifurcación
 - o Iteración
 - Indica a repetición dunha mensaxe
 - Denótase precedendo á mensaxe de *
 - o Bifurcación
 - Representa unha mensaxe cuxa execución depende da avaliación dunha condición.
 - Denótase precedendo á mensaxe dunha cláusula entre corchetes (ex. [a>b]).

Exemplos Diagramas de interacción

Diagrama de Secuencia

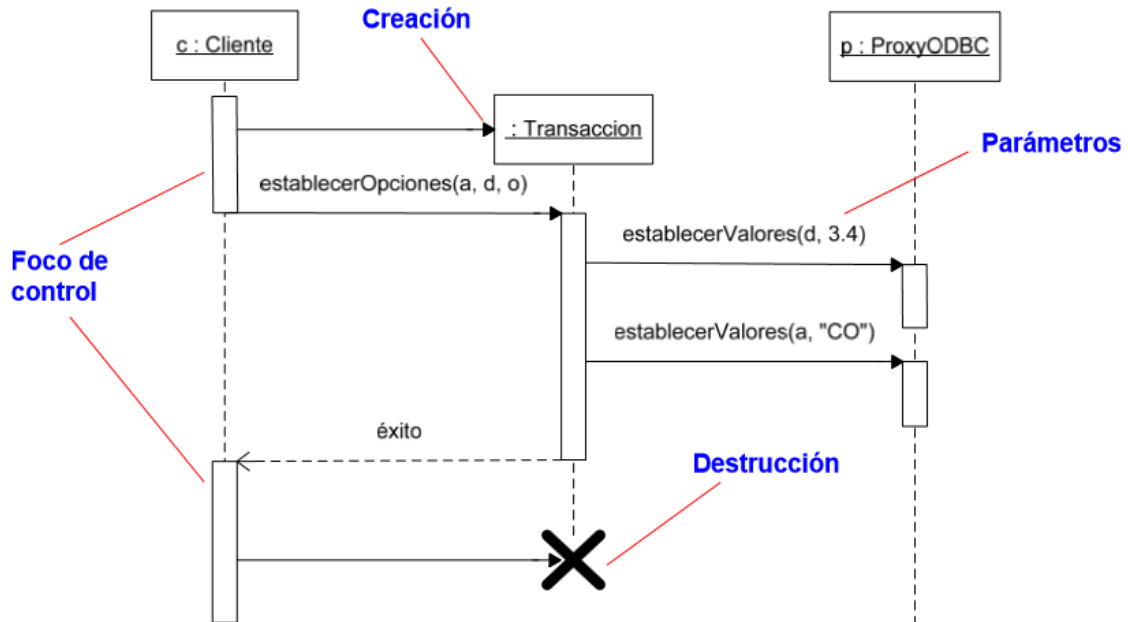
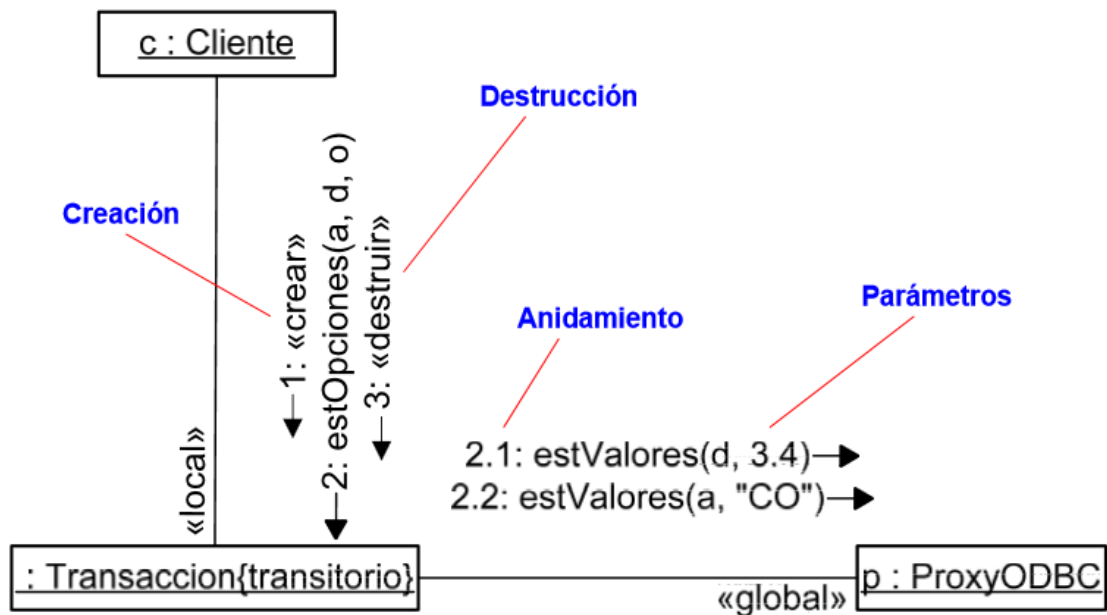


Diagrama de Colaboración



5. Modelado Estructural Avanzado

Niveis de Abstracción

- Conceptual:
 - o Representación dos conceptos do dominio

- Independente do software
- Interpretación:
 - Atributos: As asignaturas teñen códigos
 - Operacións: Especificación das responsabilidades dunha clase
- Especificación:
 - Visión de interfaces do software
 - Interpretación:
 - Atributos: As asignaturas poden dicir e establecer o seu código
 - Operacións: Especifican a interface dunha clase (operacións públicas)
- Implementación:
 - Exposición completa de clases implementadas
 - Interpretación:
 - Atributos: As asignaturas teñen un campo para gardar o seu código
 - Operacións: Contémplanse operacións privadas e protexidas

Extensibilidade

- Mecanismos de extensibilidade permiten ampliar UML
 - Estereotipos
 - Estenden o vocabulario de UML para crear novos bloques de construción
 - Aplícanse sobre bloques existentes para crear un novo
 - Especificanse con <<Estereotipo>>
 - UML ten estereotipos estándar
 - Valores etiquetados
 - Permiten engadir novas propiedades a un elemento primitivo ou a un estereotipo.
 - Aplícanse ao propio elemento, non ás instancias
 - Represéntanse entre chaves {valor=3}
 - Restricións
 - Permiten engadir nova semántica ou modificar elementos existentes
 - Especifican condicións para mellorar o modelo
 - Represéntanse entre chaves {}
 - Pódense incluír en Notas
- Serven para adaptar UML a necesidades específicas dun dominio

Clasificadores

- Mecanismo que describe características estruturais e de comportamento
- Son elementos que poden ter instancias: clases, interfaces, compoñentes, nodos, casos de uso...

Visibilidade

- Public (+)
- Protected (#)
- Private (-)
- Package (~)
- A visibilidade é pública por defecto

Alcance

- Indica se un atributo ou operación se manexa a nivel de instancia ou se é común para todas elas.
- Dous tipos de alcance
 - o De instancia
 - o De clasificador (*indícase subliñando o atributo ou operación*)

Elementos Abstractos, Raíces e follas

- Clase abstracta: non se pode instanciar (*nome en cursiva*)
- Clase folla ({leaf}): non pode ter fillos
- Clase raíz ({root}): non pode ter pais
- Operación abstracta: algún fillo da clase proporcionará unha implementación.
- Operación folla: non pode ser redefinida por ningún fillo (*nin se pode empregar polimorfismo sobre ela*)

Multiplicidade

- Multiplicidade de clase: nº de instancias que pode ter esa clase (*indefinido por omisión*)
 - o Multiplicidade 0: clase utilidade (<<utility>>). Só atributos e operacións con alcance de clasificador
 - o Multiplicidade 1: clase unitaria (<<singleton>>)
- Móstrase na esquina superior dereita do compartimento superior da clase
- Multiplicidade de atributo: rango entre corchetes xunto ao nome do atributo (atributo[*])

Asociacións

- Relacións de tipo estrutural
- Adornos básicos: nome, roles, multiplicidades e agregación
- Outras propiedades
 - o Navegabilidade
 - Dada unha asociación simple a navegación é posible en ambos sentidos da mesma
 - En ocasións pódese limitar a navegación establecendo unha frecha que indique a navegabilidade desexada
 - o Visibilidade dos roles
 - Pode limitarse a visibilidade dun rol dende obxectos externos á asociación
 - Catro niveis de visibilidade
 - Privada: obxectos do extremo non accesibles
 - Protexida: obxectos do extremo só accesibles aos obxectos fillos do outro extremo
 - Pública: comportamento por defecto

- Paquete: obxectos do extremo accesibles a obxectos do mesmo paquete
- Composición
 - Agregación con forte relación de pertenza
 - As partes poden crearse despois ou eliminarse antes pero só existen sempre que exista o “todo”
 - O “todo” é o encargado de crear e destruír as partes
 - Un obxecto só pode formar parte dun “todo”
- Clase Asociación
 - Unha asociación pode ter propiedades
 - Clase asociación: elemento con propiedades de asociación e de clase

Interfaces

- Colección de operación para especificar un servizo que proporciona unha clase ou un compoñente
- Fronteira entre comportamento desexado para unha abstracción e a implementación
- Non especifican estrutura nin implementación
- Participación en relacións
 - Unha interface pode participar en relacións de xeneralización, asociación, dependencia e realización
 - Realización: relación semántica entre dous clasificadores na que un fai uso dun contrato que outro garante
 - Unha interface especifica un contrato para unha clase ou un compoñente sen indicar a súa implementación
- Unha clase ou compoñente pode realizar moitas interfaces
- Tamén pode depender de varias interfaces

6. Padróns de Deseño

Padróns

- Solucións a problemas de deseño comúns
- Tipos:
 - Padróns de deseño (*mecanismos*)
 - Padróns de arquitectura (*frameworks*)
 - Expresan un esquema de organización estrutural para un sistema

Padróns de deseño

- “Cada patrón describe un problema recorrente nun entorno dado e a súa solución”
- Mostran estrutura e comportamento dunha sociedade de clases
- Representanse como colaboracións con aspectos:
 - Estruturais (*diagramas de clases*)
 - De comportamento (*diagramas de interacción*)
- Reutilización
 - O deseño debe adecuarse a requisitos ou problemas futuros
 - Permite reutilizar solucións

- Os patróns son flexibles e reutilizables
- Elementos:
 - Nome : describe o problema de deseño
 - Problema: contexto de aplicación
 - Solución: elementos do deseño e as súas relacións e responsabilidades
 - Consecuencias: vantaxes e inconvenientes de aplicar o patrón
- Clasificación:
 - Segundo o seu propósito:
 - De creación
 - Encárganse do proceso de creación de obxectos
 - **Abstract Factory**
 - **Factory Method**
 - **Singleton**
 - Estruturais
 - Tratan da asociación de clases ou obxectos
 - **Adapter**
 - **Composite**
 - **Decorator**
 - De comportamento
 - Caracterizan a interacción e o reparto de responsabilidades entre clases e obxectos
 - **Observer**
 - **Strategy**
 - **Template Method**
 - Segundo o seu ámbito
 - De clase
 - Encárganse de relacións entre clases e subclases fixadas en tempo de compilación
 - De obxecto
 - Tratan con relacións entre obxectos que poden cambiar en tempo de execución

Patrón de arquitectura Modelo-Vista-Controlador (MVC)

- Tríada de clases
 - Modelo: obxecto de aplicación
 - Vista: representación do modelo
 - Controlador: resposta da interface á entrada do usuario
- Vistas e modelos desacoplados empregando un protocolo de subscrición/notificación
- As visas pódense anidar
- A resposta dunha vista pode variar sen alterarse a súa representación visual

7. Patróns de Creación

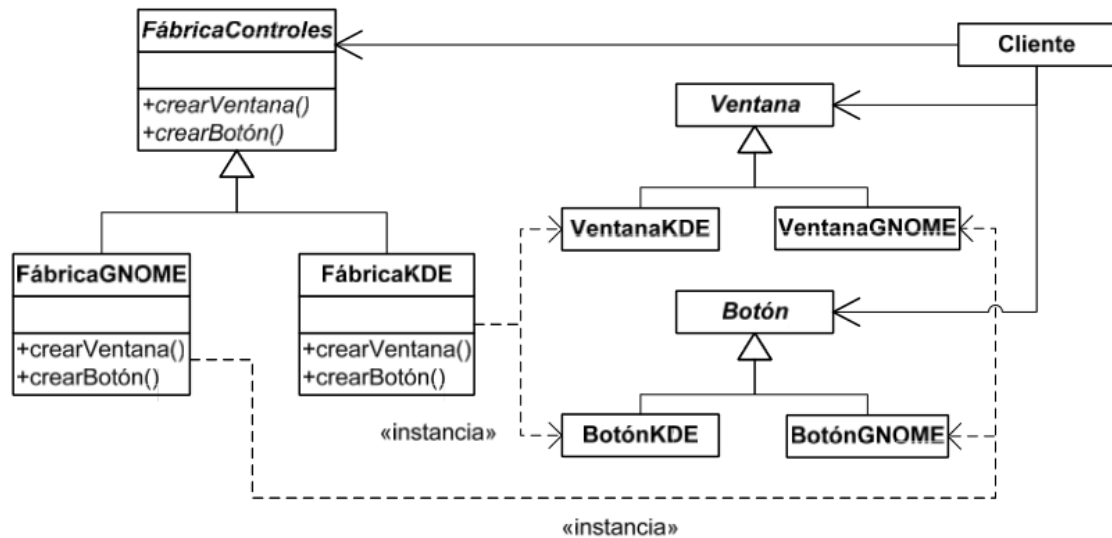
Intro

- Abstraen o proceso de creación de obxectos
- Tipos:
 - o Patróns de clases: empregan herdanza para cambiar a clase da instancia a crear
 - o Patróns de obxectos: delegan a creación de obxectos noutro obxecto
- Encapsulan coñecemento sobre clases concretas que emprega o sistema
- Ocultan como se crean e enlazan as instancias das clases
- O resto do sistema só coñece obxectos a través de interfaces
- Permiten configurar o sistema con obxectos produto que varían en estrutura e funcionalidade.

Abstract Factory

- Aporta unha interface para crear familias de obxectos ocultando clases concretas.
- Emprégase:
 - o Se un sistema debe ser configurado para unha familia de produtos de entre varias
 - o Se cada familia está empregada para ser usada conxuntamente
 - o Para proporcionar librerías de produtos dos que só se revelan as interfaces
- Participantes:
 - o AbstractFactory: declara unha interface para operacións que crean produtos abstractos.
 - o ConcreteFactory: implementa operacións para crear produtos concretos.
 - o AbstractProduct: declara unha interface para un tipo de produto
 - o ConcreteProduct: define un produto a ser creado pola fábrica correspondente.
 - o Cliente: usa interfaces de clases abstractas.
- Vantaxes e inconvenientes:
 - o Illamento entre clientes e clases de implementación.
 - o Facilitade de cambio de familias
 - o Consistencia: as aplicacións só xeran produtos da mesma familia
 - o Dificultade de introdución de novos produtos

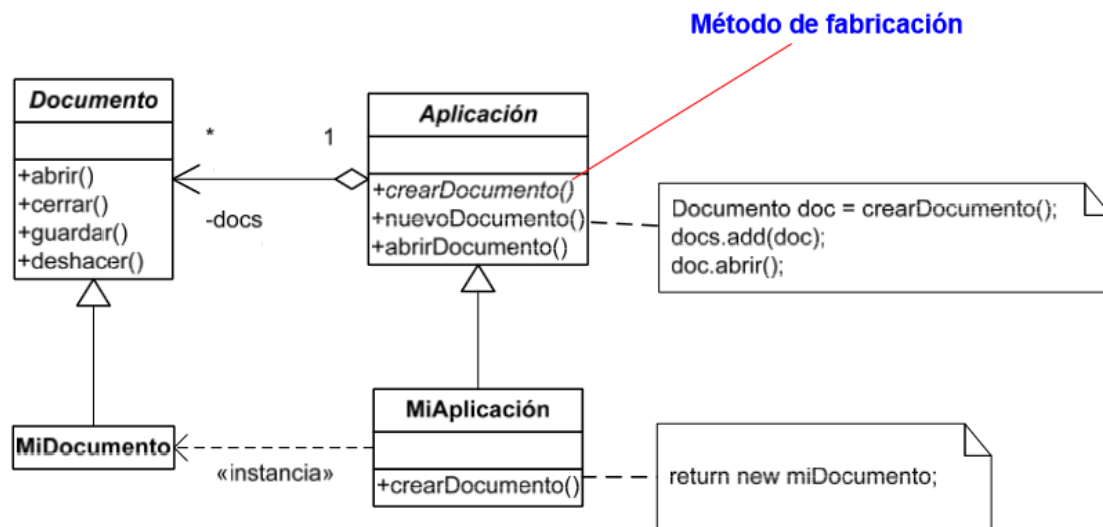
Exemplo Abstract Factory



Factory Method

- Define unha interface para crear obxectos deixando a subclases elixir que clase instanciar
- Permite que unha clase delegue nas súas subclases a creación de obxectos
- Emprégase:
 - o Unha clase non pode prever a clase de obxectos que debe crear
 - o Unha clase quere delegar nas súas clases a creación de obxectos
- Participantes:
 - o Produto: define a interface dos obxectos creados polo Factory Method
 - o ProdutoConcreto: implementa a interface Produto
 - o Creador: declara (*e pode implementar*) o método de fabricación, o cal devolve un obxecto Produto
 - o CreadorConcreto: redefine o método de fabricación para devolver unha instancia de ProdutoConcreto
- Vantaxes e inconvenientes:
 - o O código do usuario só traballa coa interface produto
 - o Os clientes poden ter que herdar do Creador simplemente para crear un ProdutoConcreto
 - o Dota a subclase de enganche para proveer unha versión estendida dun obxecto.
 - o Pode conectar xerarquías paralelas

Exemplo Factory Method



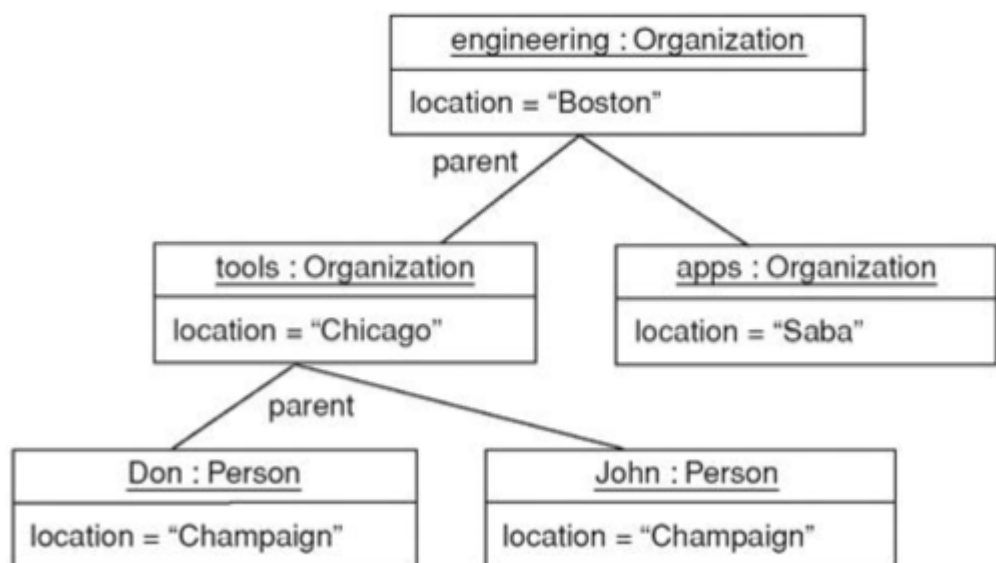
8. Patrones Estructurais

Mirar en Gradox o PDF con Patrones

9. BONUS

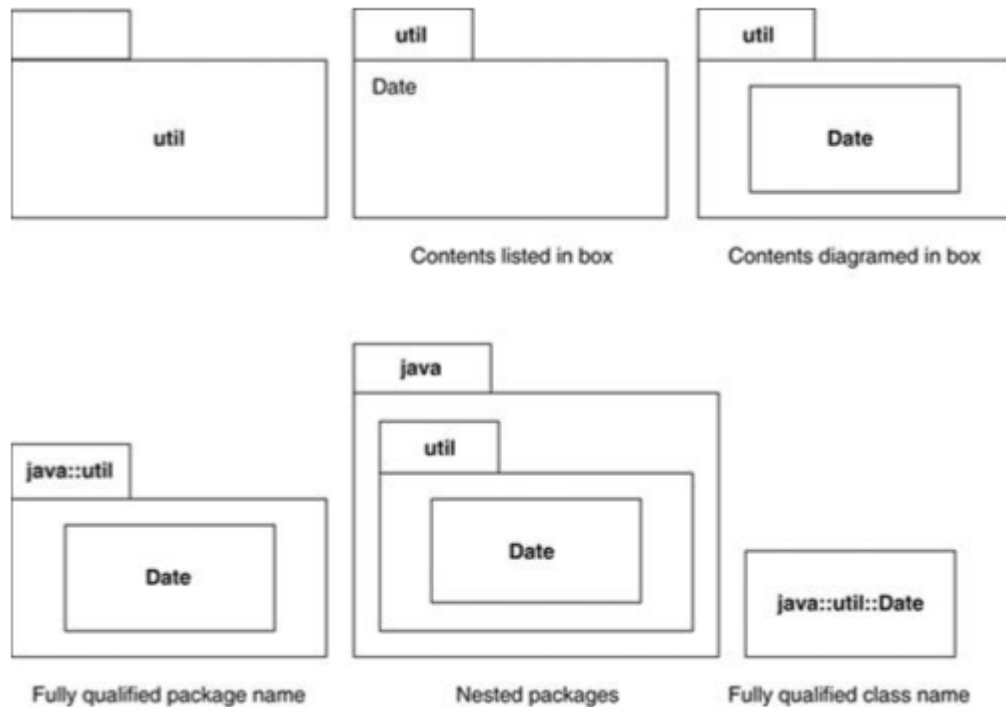
Diagramas de Obxectos

- An object diagram is a snapshot of the objects in a system at a point in time. Because it shows instances rather than classes, an object diagram is often called an instance diagram.



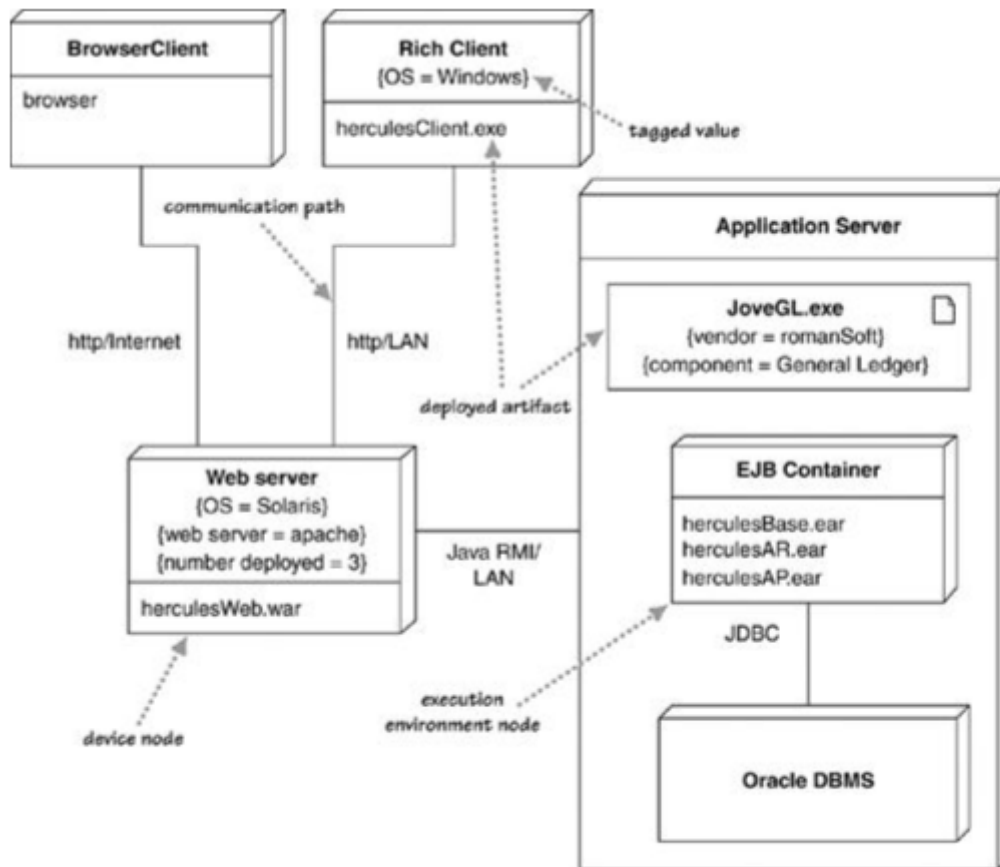
Diagramas de Paquetes

- A package is a grouping construct that allows you to take any construct in the UML and group its elements together into higher-level units. Its most common use is to group classes, and that's the way I'm describing it here, but remember that you can use packages for every other bit of the UML as well.
- The UML allows classes in a package to be public or private. A public class is part of the interface of the package and can be used by classes in other packages; a private class is hidden.



Diagramas de “Despliegue”

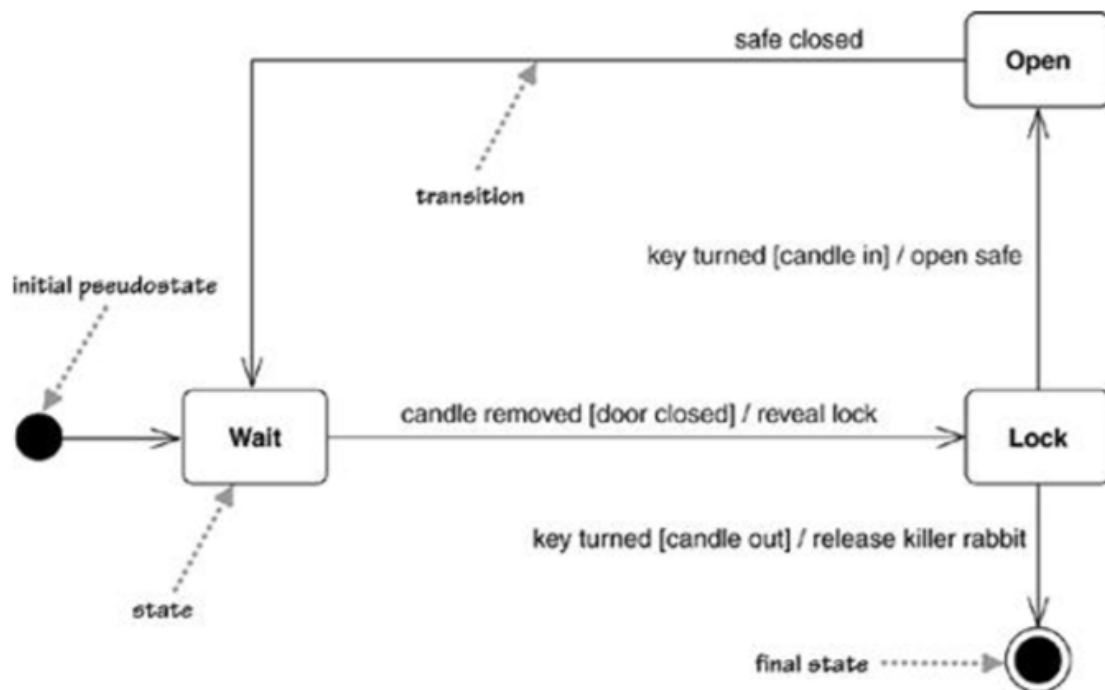
- Deployment diagrams show a system's physical layout, revealing which pieces of software run on what pieces of hardware.
- The main items on the diagram are nodes connected by communication paths.
 - o A node is something that can host some software. Nodes come in two forms.
 - A device is hardware, it may be a computer or a simpler piece of hardware connected to a system.
 - An execution environment is software that itself hosts or contains other software, examples are an operating system or a container process.
 - o The nodes contain artifacts, which are the physical manifestations of software: usually, files.



Diagramas de Estado (*State Machine Diagrams*)

- State machine diagrams are a familiar technique to describe the behavior of a system.
 - o The controller can be in three states: Wait, Lock, and Open
 - o The transition indicates a movement from one state to another.
 - Each transition has a label that comes in three parts: trigger-signature [guard]/activity.
 - All the parts are optional.
 - A missing activity indicates that you don't do anything during the transition. A missing guard indicates that you always take the transition if the event occurs. A missing trigger-signature is rare but does occur. It indicates that you take the transition immediately.
- Internal Activities
 - o States can react to events without transition, using internal activities: putting the event, guard, and activity inside the state box itself.
- Activity States
 - o You can have states in which the object is doing some ongoing work.
 - o The ongoing activity is marked with the **do/**.
- Superstates

- Often, you'll find that several states share common transitions and internal activities. In these cases, you can make them substates and move the shared behavior into a superstate.
- Concurrent States
 - States can be broken into several orthogonal state diagrams that run concurrently.



Diagramas de Actividade

- Activity diagrams are a technique to describe procedural logic, business process, and work flow. In many ways, they play a role similar to flowcharts, but the principal difference between them and flowchart notation is that they support parallel behavior.
- Signals
 - A signal indicates that the activity receives an event from an outside process.
- Flows and Edges
 - UML 2 uses the terms flow and edge synonymously to describe the connections between two actions. The simplest kind of edge is the simple arrow between two actions. You can give an edge a name if you like, but most of the time, a simple arrow will suffice.
- Expansion Regions
 - An expansion region marks an activity diagram area where actions occur once for each item in a collection.
- Flow Final
 - Once you get multiple tokens, as in an expansion region, you often get flows that stop even when the activity as a whole doesn't end. A flow final

indicates the end of one particular flow, without terminating the whole activity.

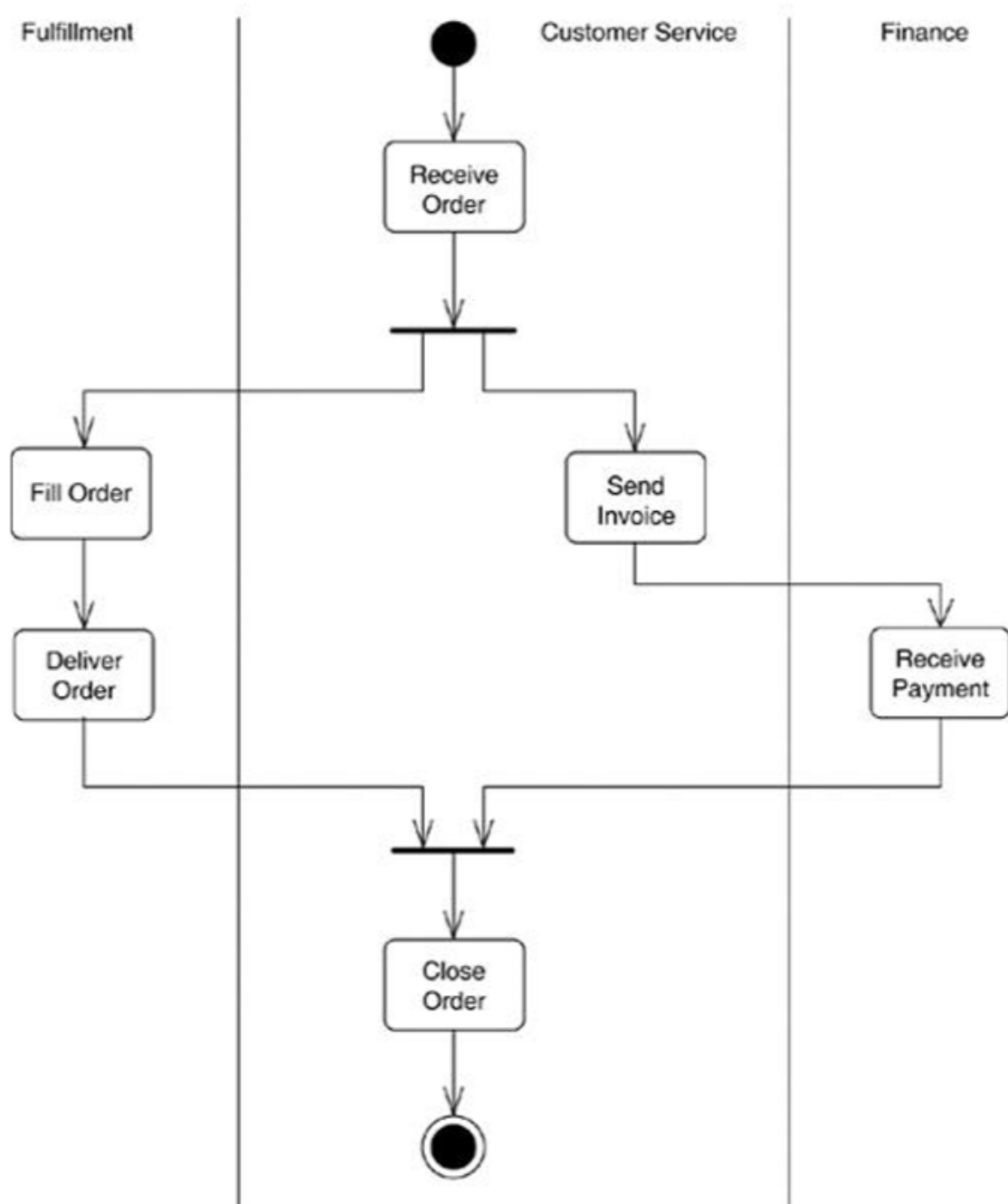


Diagrama de Componentes

- Components are not a technology. Technology people seem to find this hard to understand. Components are about how customers want to relate to software. They want to be able to buy their software a piece at a time, and to be able to upgrade it just like they can upgrade their stereo. They want new pieces to work seamlessly with their old pieces, and to be able to upgrade on their own schedule, not the manufacturer's schedule. They want to be able to mix and match pieces

from various manufacturers. This is a very reasonable requirement. It is just hard to satisfy.

